Big Data Assignment 2 — Report

---

1. Methodology

1.1 System Overview

The goal of this assignment was to build a distributed search engine for plain text documents. The system takes a corpus of 100 documents, indexes them using Hadoop MapReduce, stores the resulting index in Cassandra, and answers free-text queries using BM25 ranking. The full pipeline runs inside Docker Compose and requires no manual setup steps beyond `docker compose up`.

The pipeline uses distributed tools at every major stage: PySpark for data preparation and query execution, Hadoop MapReduce for index construction, and Cassandra as the storage and retrieval layer. No single-machine libraries such as pandas were used at any point. Data preparation includes a local driver-based staging step before uploading to HDFS; all subsequent stages — indexing, storage, and search — run fully distributed on the cluster.

1.2 Cluster Architecture

The cluster is defined with Docker Compose and consists of three containers:

- **`cluster-master`** — orchestrates the full workflow: starts Hadoop services, prepares the data with PySpark, launches MapReduce jobs, loads data into Cassandra, and submits the search job to YARN.
- **`cluster-slave-1`** — acts as the Hadoop worker node, running YARN NodeManager and HDFS DataNode. MapReduce tasks and YARN executors are scheduled here.
- **`cassandra-server`** — stores the final inverted index and serves low-latency lookups during query time.

Both `cluster-master` and `cluster-slave-1` use the same Docker image. This ensures that the Python runtime and all dependencies are identical on both nodes, which is important for YARN-distributed PySpark execution.

1.3 Data Preparation (`prepare_data.py`)

Data preparation is implemented as a PySpark job. The primary data source is a Parquet file at `/a.parquet` in HDFS, containing three columns: `id`, `title`, and `text`. If that file is not found, the pipeline falls back to bundled plain text files in `app/data`.

PySpark reads the source, drops rows where `text` is null or empty, and samples exactly 100 documents. For each selected document, the script generates a local `.txt` file named in the format `<doc_id>_<doc_title>.txt`. These files are uploaded to HDFS under `/data/`, then read back with Spark and written to `/input/data` as a single-partition text file in tab-separated format:

```
doc_id<TAB>title<TAB>text
```

This file is the input for the first MapReduce pipeline.

Design note: this stage is a hybrid preprocessing step. PySpark is used for all reading, filtering, and sampling — no pandas or single-machine data processing libraries are involved. However, Spark runs in local mode here, and the driver writes local `.txt` files before uploading them to HDFS. This approach was chosen for simplicity and stability in the Docker environment and does not affect the correctness of the downstream distributed pipeline.

1.4 Index Construction (`create_index.sh`)

The index is built with two sequential MapReduce jobs.

Pipeline 1 — Inverted Index (`mapper1.py` / `reducer1.py`)

The mapper reads each line of `/input/data`, tokenizes the document text using alphanumeric tokens, and emits two types of records:

- A **document metadata record** per document:

  ```
  __DOC__{doc_id}    DOC    {doc_id}    {title}    {doc_length}
  ```

  where `doc_length` is the total token count.

- A **posting record** per distinct term:

  ```
  {term}    POSTING    {doc_id}    {tf}    {title}    {doc_length}
  ```

  where `tf` is term frequency within the document.

The reducer groups records by key. For each term it emits one vocabulary row (`VOCAB {term} {df}`) and one posting row per matching document. Document metadata rows are passed through unchanged.

After the MapReduce job completes, `create_index.sh` splits the output into three separate TSV files in HDFS:

HDFS path	Content
`/indexer/postings`	`term, doc_id, tf, title, doc_length`
`/indexer/vocabulary`	`term, df`
`/indexer/documents`	`doc_id, title, doc_length`

Pipeline 2 — Corpus Statistics (`mapper2.py` / `reducer2.py`)

The second job reads the `DOC` records produced by Pipeline 1. Each document contributes `1` to the document count and its `doc_length` to the total token count. The reducer computes:

- `N` — total number of documents
- `AVGDL` — average document length in tokens

Results are stored at `/indexer/stats` in the format:

```
STAT    N       100
STAT    AVGDL   <value computed from the indexed corpus>
```

These values are required by the BM25 scoring formula.

1.5 Cassandra Storage (`app.py`, `store_index.sh`)

Once the HDFS index is ready, all four datasets are loaded into Cassandra under the keyspace `search_engine` (SimpleStrategy, replication factor 1). The loader script truncates all tables on each run and reloads them from HDFS, which makes the pipeline idempotent.

The schema is as follows:

```cpp
CREATE TABLE documents (
    doc_id text PRIMARY KEY,
    title text,
    doc_length int
);

CREATE TABLE vocabulary (
    term text PRIMARY KEY,
    df int
);

CREATE TABLE postings (
    term text,
    doc_id text,
    tf int,
    title text,
    doc_length int,
    PRIMARY KEY (term, doc_id)
);

CREATE TABLE corpus_stats (
    stat_name text PRIMARY KEY,
    stat_value double
);
```

The `postings` table uses a composite primary key `(term, doc_id)`, which allows Cassandra to efficiently retrieve all documents containing a given term — exactly the access pattern needed at query

time.

Example rows from a sample run:

- `documents`: `("10174562", "A History of Money and Banking in the United States", 1234)`
- `vocabulary`: `("money", 58)`
- `postings`: `("money", "10174562", 4, "A History of Money and Banking in the United States", 1234)`
- `corpus_stats`: `("AVGDL", 842.61)` *(example value from one run)*

1.6 BM25 Search (`query.py`, `search.sh`)

Search queries are executed by a PySpark job submitted to YARN in client mode. The job connects to Cassandra, retrieves the relevant index data, and ranks documents using the BM25 formula.

BM25 parameters (defined in `engine_utils.py`):

- `k1 = 1.2`
- `b = 0.75`

Scoring formula for each (term, document) pair:

```
idf  = log(N / df)
norm = k1 * ((1 - b) + b * (doc_length / AVGDL))
score = idf * (tf * (k1 + 1)) / (tf + norm)
```

Query flow:

1. The query string is taken from the command line.
2. It is tokenized with the regex `[A-Za-z0-9]+` and lowercased.
3. The app connects to `cassandra-server` and reads `N` and `AVGDL` from `corpus_stats`.
4. For each query term, `df` is looked up in `vocabulary` and all matching rows are fetched from `postings`.
5. Each row becomes a tuple `(doc_id, title, tf, doc_length, df)`, loaded into a Spark RDD.
6. BM25 scores are computed in parallel; `reduceByKey` sums scores across terms for each document.
7. `takeOrdered(10, key=lambda x: -x[2])` returns the top 10 results.
8. Results are printed as `doc_id<TAB>title`.

The `search.sh` script submits the job to YARN in client mode:

```
spark-submit \
  --master yarn \
  --deploy-mode client \
  --driver-memory 512m \
  --executor-memory 512m \
  --py-files /app/engine_utils.py \
  query.py "$@"
```

Client mode was chosen over cluster mode because it keeps the driver on the master node, which simplifies log access and makes the pipeline easier to monitor and grade in this Docker environment. The executors still run on the YARN cluster, so the execution remains distributed.

---

2. Demonstration

2.1 How to Run

Make sure Docker Desktop is running. Clone the repository and start the full pipeline with:

```
docker compose up --build
```

The master container will automatically run every step in sequence:

1. Start Hadoop and Cassandra services
2. Prepare 100 documents and upload them to HDFS
3. Build the inverted index with two MapReduce jobs
4. Load the index into Cassandra
5. Execute sample BM25 search queries and print results

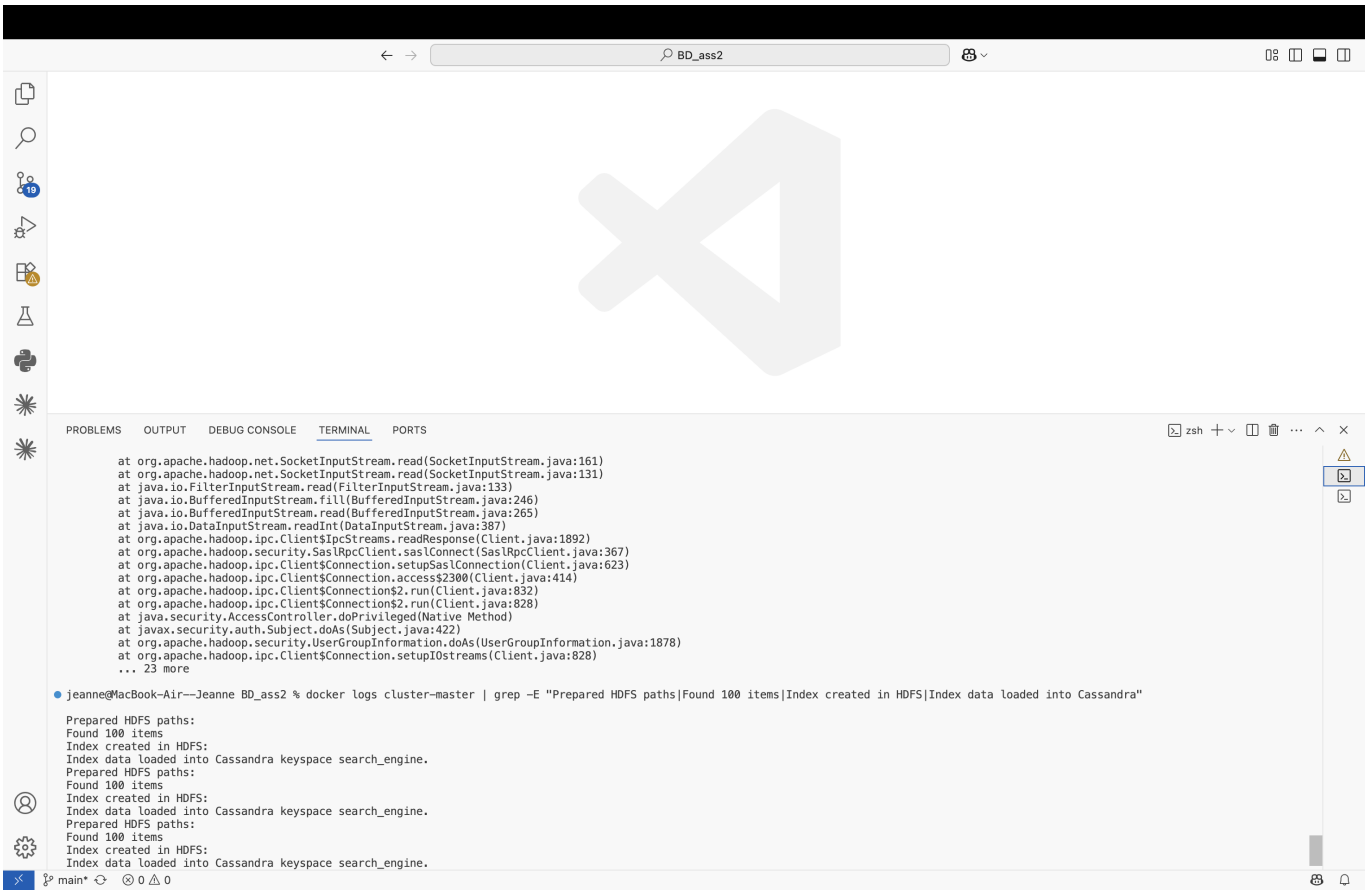
No manual steps are required. The pipeline finishes when you see:

```
SparkContext is stopping with exitCode 0.
cluster-master exited with code 0
```

To run a custom query after the pipeline has finished:

```
docker exec cluster-master bash /app/search.sh "your query here"
```

2.2 Screenshot 1 — Successful Indexing of 100 Documents



The screenshot shows the log lines confirming that all indexing stages completed successfully, including `Prepared HDFS paths`, `Found 100 items`, `Index created in HDFS`, and `Index data loaded into Cassandra keyspace search_engine`.

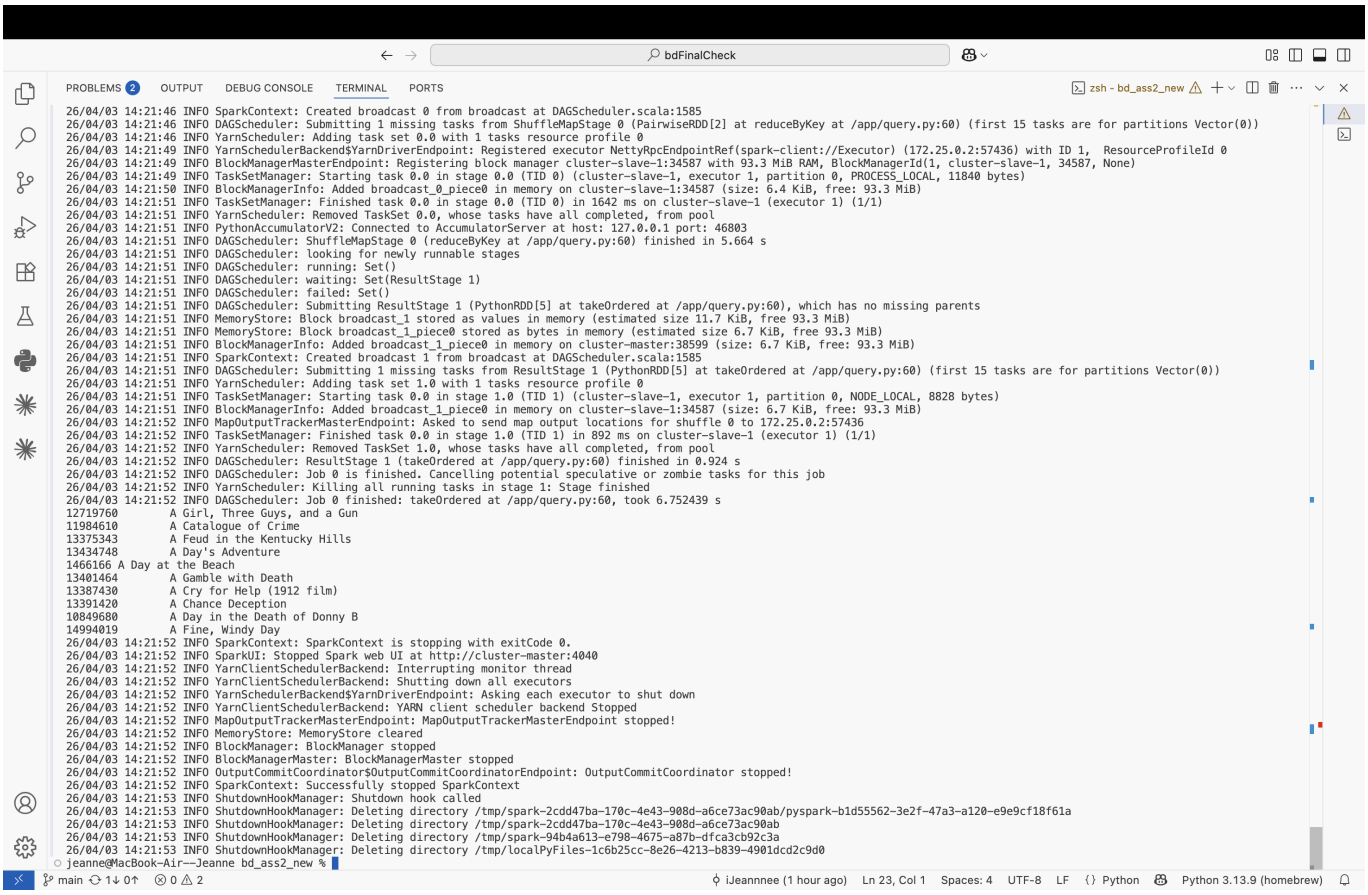
2.3 Screenshot 2 — Search Query 1 - "history money banking"

```
26/04/03 14:13:06 INFO MemoryStore: Block broadcast_0_piece0 stored as bytes in memory (estimated size 6.4 KiB, free 93.3 MiB)
26/04/03 14:13:06 INFO BlockManagerInfo: Added broadcast_0_piece0 in memory on cluster-master:40205 (size: 6.4 KiB, free: 93.3 MiB)
26/04/03 14:13:06 INFO SparkContext: Created broadcast 0 from broadcast at DAGScheduler.scala:1585
26/04/03 14:13:06 INFO DAGScheduler: Submitting 1 missing tasks from ShuffleMapStage 0 (PairwiseRDD[2] at reduceByKey at /app/query.py:60) (first 15 tasks are for partitions Vector(0))
26/04/03 14:13:06 INFO YarnScheduler: Adding task set 0.0 with 1 tasks resource profile 0
26/04/03 14:13:06 INFO TaskSetManager: Starting task 0.0 in stage 0.0 (TID 0) (cluster-slave-1, executor 1, partition 0, PROCESS_LOCAL, 10433 bytes)
26/04/03 14:13:07 INFO BlockManagerInfo: Added broadcast_0_piece0 in memory on cluster-slave-1:44065 (size: 6.4 KiB, free: 93.3 MiB)
26/04/03 14:13:08 INFO TaskSetManager: Finished task 0.0 in stage 0.0 (TID 0) in 1656 ms on cluster-slave-1 (executor 1) (1/1)
26/04/03 14:13:08 INFO YarnScheduler: Removed TaskSet 0.0, whose tasks have all completed, from pool
26/04/03 14:13:08 INFO PythonAccumulatorV2: Connected to AccumulatorServer at host: 127.0.0.1 port: 50471
26/04/03 14:13:08 INFO DAGScheduler: ShuffleMapStage 0 (reduceByKey at /app/query.py:60) finished in 1.932 s
26/04/03 14:13:08 INFO DAGScheduler: looking for newly runnable stages
26/04/03 14:13:08 INFO DAGScheduler: running: Set()
26/04/03 14:13:08 INFO DAGScheduler: waiting: Set(ResultStage 1)
26/04/03 14:13:08 INFO DAGScheduler: failed: Set()
26/04/03 14:13:08 INFO DAGScheduler: Submitting ResultStage 1 (PythonRDD[5] at takeOrdered at /app/query.py:60), which has no missing parents
26/04/03 14:13:08 INFO MemoryStore: Block broadcast_1 stored as values in memory (estimated size 11.7 KiB, free 93.3 MiB)
26/04/03 14:13:08 INFO MemoryStore: Block broadcast_1_piece0 stored as bytes in memory (estimated size 6.7 KiB, free 93.3 MiB)
26/04/03 14:13:08 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on cluster-master:40205 (size: 6.7 KiB, free: 93.3 MiB)
26/04/03 14:13:08 INFO SparkContext: Created broadcast 1 from broadcast at DAGScheduler.scala:1585
26/04/03 14:13:08 INFO DAGScheduler: Submitting 1 missing tasks from ResultStage 1 (PythonRDD[5] at takeOrdered at /app/query.py:60) (first 15 tasks are for partitions Vector(0))
26/04/03 14:13:08 INFO YarnScheduler: Adding task set 1.0 with 1 tasks resource profile 0
26/04/03 14:13:08 INFO TaskSetManager: Starting task 0.0 in stage 1.0 (TID 1) (cluster-slave-1, executor 1, partition 0, NODE_LOCAL, 8828 bytes)
26/04/03 14:13:08 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on cluster-slave-1:44065 (size: 6.7 KiB, free: 93.3 MiB)
26/04/03 14:13:09 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 0 to 172.25.0.2:37064
26/04/03 14:13:09 INFO TaskSetManager: Finished task 0.0 in stage 1.0 (TID 1) in 608 ms on cluster-slave-1 (executor 1) (1/1)
26/04/03 14:13:09 INFO YarnScheduler: Removed TaskSet 1.0, whose tasks have all completed, from pool
26/04/03 14:13:09 INFO DAGScheduler: ResultStage 1 (takeOrdered at /app/query.py:60) finished in 0.677 s
26/04/03 14:13:09 INFO DAGScheduler: Job 0 is finished. Cancelling potential speculative or zombie tasks for this job
26/04/03 14:13:09 INFO YarnScheduler: Killing all running tasks in stage 1: Stage finished
26/04/03 14:13:09 INFO DAGScheduler: Job 0 finished: takeOrdered at /app/query.py:60, took 2.769113 s
10174562        A History of Money and Banking in the United States
14782494        A Countess Below Stairs
10399316        A Flowering Tree
10381993        A Doll's House (1973 Losey film)
13958799        A Dying Colonialism
1083442 A Hillbilly Tribute to ACDC
13060654        A Lion Among Men
10849680        A Day in the Death of Donny B
15433209        A Distant Trumpet
12666064        A Bird in the House
26/04/03 14:13:09 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/04/03 14:13:09 INFO SparkUI: Stopped Spark web UI at http://cluster-master:4040
26/04/03 14:13:09 INFO YarnClientSchedulerBackend: Interrupting monitor thread
26/04/03 14:13:09 INFO YarnClientSchedulerBackend: Shutting down all executors
26/04/03 14:13:09 INFO YarnSchedulerBackend$YarnDriverEndpoint: Asking each executor to shut down
26/04/03 14:13:09 INFO YarnClientSchedulerBackend: YARN client scheduler backend Stopped
26/04/03 14:13:09 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/04/03 14:13:09 INFO MemoryStore: MemoryStore cleared
26/04/03 14:13:09 INFO BlockManager: BlockManager stopped
26/04/03 14:13:09 INFO BlockManagerMaster: BlockManagerMaster stopped
26/04/03 14:13:09 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/04/03 14:13:09 INFO SparkContext: Successfully stopped SparkContext
26/04/03 14:13:10 INFO ShutdownHookManager: Shutdown hook called
26/04/03 14:13:10 INFO ShutdownHookManager: Deleting directory /tmp/spark-1e5ca6f1-23a0-4997-a3be-725faa0fb7a9/pyspark-90791199-d2d5-40b9-9752-396caa3c9146
26/04/03 14:13:10 INFO ShutdownHookManager: Deleting directory /tmp/localPyFiles-46c8ecf8-4f54-445c-a164-4e202fbeb564
26/04/03 14:13:10 INFO ShutdownHookManager: Deleting directory /tmp/spark-1e5ca6f1-23a0-4997-a3be-725faa0fb7a9
26/04/03 14:13:10 INFO ShutdownHookManager: Deleting directory /tmp/spark-d2415a02-179b-45fe-84b3-e9e764638de5
jeanne@MacBook-Air--Jeanne bd_ass2_new %
```

2.4 Screenshot 3 — Search Query 2 - "sherlock holmes"

```
26/04/03 14:14:43 INFO BlockManagerMasterEndpoint: Registering block manager cluster-slave-1:42789 with 93.3 MiB RAM, BlockManagerId(1, cluster-slave-1, 42789, None)
26/04/03 14:14:44 INFO SparkContext: Starting job: takeOrdered at /app/query.py:60
26/04/03 14:14:44 INFO DAGScheduler: Registering RDD 2 (reduceByKey at /app/query.py:60) as input to shuffle 0
26/04/03 14:14:44 INFO DAGScheduler: Got job 0 (takeOrdered at /app/query.py:60) with 1 output partitions
26/04/03 14:14:44 INFO DAGScheduler: Final stage: ResultStage 1 (takeOrdered at /app/query.py:60)
26/04/03 14:14:44 INFO DAGScheduler: Parents of final stage: List(ShuffleMapStage 0)
26/04/03 14:14:44 INFO DAGScheduler: Missing parents: List(ShuffleMapStage 0)
26/04/03 14:14:44 INFO DAGScheduler: Submitting ShuffleMapStage 0 (PairwiseRDD[2] at reduceByKey at /app/query.py:60), which has no missing parents
26/04/03 14:14:44 INFO MemoryStore: Block broadcast_0 stored as values in memory (estimated size 9.9 KiB, free 93.3 MiB)
26/04/03 14:14:44 INFO MemoryStore: Block broadcast_0_piece0 stored as bytes in memory (estimated size 6.4 KiB, free 93.3 MiB)
26/04/03 14:14:44 INFO BlockManagerInfo: Added broadcast_0_piece0 in memory on cluster-master:39055 (size: 6.4 KiB, free: 93.3 MiB)
26/04/03 14:14:44 INFO SparkContext: Created broadcast 0 from broadcast at DAGScheduler.scala:1585
26/04/03 14:14:44 INFO DAGScheduler: Submitting 1 missing tasks from ShuffleMapStage 0 (PairwiseRDD[2] at reduceByKey at /app/query.py:60) (first 15 tasks are for partitions Vector(0))
26/04/03 14:14:44 INFO YarnScheduler: Adding task set 0.0 with 1 tasks resource profile 0
26/04/03 14:14:44 INFO TaskSetManager: Starting task 0.0 in stage 0.0 (TID 0) (cluster-slave-1, executor 1, partition 0, PROCESS_LOCAL, 9135 bytes)
26/04/03 14:14:44 INFO BlockManagerInfo: Added broadcast_0_piece0 in memory on cluster-slave-1:42789 (size: 6.4 KiB, free: 93.3 MiB)
26/04/03 14:14:46 INFO TaskSetManager: Finished task 0.0 in stage 0.0 (TID 0) in 1750 ms on cluster-slave-1 (executor 1) (1/1)
26/04/03 14:14:46 INFO YarnScheduler: Removed TaskSet 0.0, whose tasks have all completed, from pool
26/04/03 14:14:46 INFO PythonAccumulatorV2: Connected to AccumulatorServer at host: 127.0.0.1 port: 52125
26/04/03 14:14:46 INFO DAGScheduler: ShuffleMapStage 0 (reduceByKey at /app/query.py:60) finished in 1.968 s
26/04/03 14:14:46 INFO DAGScheduler: looking for newly runnable stages
26/04/03 14:14:46 INFO DAGScheduler: running: Set()
26/04/03 14:14:46 INFO DAGScheduler: waiting: Set(ResultStage 1)
26/04/03 14:14:46 INFO DAGScheduler: failed: Set()
26/04/03 14:14:46 INFO DAGScheduler: Submitting ResultStage 1 (PythonRDD[5] at takeOrdered at /app/query.py:60), which has no missing parents
26/04/03 14:14:46 INFO MemoryStore: Block broadcast_1 stored as values in memory (estimated size 11.7 KiB, free 93.3 MiB)
26/04/03 14:14:46 INFO MemoryStore: Block broadcast_1_piece0 stored as bytes in memory (estimated size 6.7 KiB, free 93.3 MiB)
26/04/03 14:14:46 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on cluster-master:39055 (size: 6.7 KiB, free: 93.3 MiB)
26/04/03 14:14:46 INFO SparkContext: Created broadcast 1 from broadcast at DAGScheduler.scala:1585
26/04/03 14:14:46 INFO DAGScheduler: Submitting 1 missing tasks from ResultStage 1 (PythonRDD[5] at takeOrdered at /app/query.py:60) (first 15 tasks are for partitions Vector(0))
26/04/03 14:14:46 INFO YarnScheduler: Adding task set 1.0 with 1 tasks resource profile 0
26/04/03 14:14:46 INFO TaskSetManager: Starting task 0.0 in stage 1.0 (TID 1) (cluster-slave-1, executor 1, partition 0, NODE_LOCAL, 8828 bytes)
26/04/03 14:14:46 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on cluster-slave-1:42789 (size: 6.7 KiB, free: 93.3 MiB)
26/04/03 14:14:46 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 0 to 172.25.0.2:33224
26/04/03 14:14:46 INFO TaskSetManager: Finished task 0.0 in stage 1.0 (TID 1) in 626 ms on cluster-slave-1 (executor 1) (1/1)
26/04/03 14:14:46 INFO YarnScheduler: Removed TaskSet 1.0, whose tasks have all completed, from pool
26/04/03 14:14:46 INFO DAGScheduler: ResultStage 1 (takeOrdered at /app/query.py:60) finished in 0.655 s
26/04/03 14:14:46 INFO DAGScheduler: Job 0 is finished. Cancelling potential speculative or zombie tasks for this job
26/04/03 14:14:46 INFO YarnScheduler: Killing all running tasks in stage 1: Stage finished
26/04/03 14:14:46 INFO DAGScheduler: Job 0 finished: takeOrdered at /app/query.py:60, took 2.789440 s
11984610        A Catalogue of Crime
15523771        A Day in the Death
26/04/03 14:14:46 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/04/03 14:14:46 INFO SparkUI: Stopped Spark web UI at http://cluster-master:4040
26/04/03 14:14:47 INFO YarnClientSchedulerBackend: Interrupting monitor thread
26/04/03 14:14:47 INFO YarnClientSchedulerBackend: Shutting down all executors
26/04/03 14:14:47 INFO YarnSchedulerBackend$YarnDriverEndpoint: Asking each executor to shut down
26/04/03 14:14:47 INFO YarnClientSchedulerBackend: YARN client scheduler backend Stopped
26/04/03 14:14:47 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/04/03 14:14:47 INFO MemoryStore: MemoryStore cleared
26/04/03 14:14:47 INFO BlockManager: BlockManager stopped
26/04/03 14:14:47 INFO BlockManagerMaster: BlockManagerMaster stopped
26/04/03 14:14:47 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/04/03 14:14:47 INFO SparkContext: Successfully stopped SparkContext
26/04/03 14:14:48 INFO ShutdownHookManager: Shutdown hook called
26/04/03 14:14:48 INFO ShutdownHookManager: Deleting directory /tmp/spark-c421c1ba-20a5-4155-807e-27d75b366926/pyspark-c40ebde2-9fa7-4a78-8aa1-ca15dca180be
26/04/03 14:14:48 INFO ShutdownHookManager: Deleting directory /tmp/localPyFiles-96f95d4d-a666-4465-9ae5-95d543fb2666
26/04/03 14:14:48 INFO ShutdownHookManager: Deleting directory /tmp/spark-dab28dd7-fb2f-4ca4-8b9d-e1b3b106e97b
26/04/03 14:14:48 INFO ShutdownHookManager: Deleting directory /tmp/spark-c421c1ba-20a5-4155-807e-27d75b366926
```

2.5 Screenshot 4 — Search Query 3 - "crime drama film"



```
26/04/03 14:21:46 INFO SparkContext: Created broadcast 0 from broadcast at DAGScheduler.scala:1585
26/04/03 14:21:46 INFO DAGScheduler: Submitting 1 missing tasks from ShuffleMapStage 0 (PairwiseRDD[2] at reduceByKey at /app/query.py:60) (first 15 tasks are for partitions Vector(0))
26/04/03 14:21:46 INFO YarnScheduler: Adding task set 0.0 with 1 tasks resource profile 0
26/04/03 14:21:49 INFO YarnSchedulerBackend$YarnDriverEndpoint: Registered executor NettyRpcEndpointRef(spark-client://Executor) (172.25.0.2:57436) with ID 1,  ResourceProfileId 0
26/04/03 14:21:49 INFO BlockManagerMasterEndpoint: Registering block manager cluster-slave-1:34587 with 93.3 MiB RAM, BlockManagerId(1, cluster-slave-1, 34587, None)
26/04/03 14:21:49 INFO TaskSetManager: Starting task 0.0 in stage 0.0 (TID 0) (cluster-slave-1, executor 1, partition 0, PROCESS_LOCAL, 11840 bytes)
26/04/03 14:21:50 INFO BlockManagerInfo: Added broadcast_0_piece0 in memory on cluster-slave-1:34587 (size: 6.4 KiB, free: 93.3 MiB)
26/04/03 14:21:51 INFO TaskSetManager: Finished task 0.0 in stage 0.0 (TID 0) in 1642 ms on cluster-slave-1 (executor 1) (1/1)
26/04/03 14:21:51 INFO YarnScheduler: Removed TaskSet 0.0, whose tasks have all completed, from pool
26/04/03 14:21:51 INFO PythonAccumulatorV2: Connected to AccumulatorServer at host: 127.0.0.1 port: 46803
26/04/03 14:21:51 INFO DAGScheduler: ShuffleMapStage 0 (reduceByKey at /app/query.py:60) finished in 5.664 s
26/04/03 14:21:51 INFO DAGScheduler: looking for newly runnable stages
26/04/03 14:21:51 INFO DAGScheduler: running: Set()
26/04/03 14:21:51 INFO DAGScheduler: waiting: Set(ResultStage 1)
26/04/03 14:21:51 INFO DAGScheduler: failed: Set()
26/04/03 14:21:51 INFO DAGScheduler: Submitting ResultStage 1 (PythonRDD[5] at takeOrdered at /app/query.py:60), which has no missing parents
26/04/03 14:21:51 INFO MemoryStore: Block broadcast_1 stored as values in memory (estimated size 11.7 KiB, free 93.3 MiB)
26/04/03 14:21:51 INFO MemoryStore: Block broadcast_1_piece0 stored as bytes in memory (estimated size 6.7 KiB, free 93.3 MiB)
26/04/03 14:21:51 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on cluster-master:38599 (size: 6.7 KiB, free: 93.3 MiB)
26/04/03 14:21:51 INFO SparkContext: Created broadcast 1 from broadcast at DAGScheduler.scala:1585
26/04/03 14:21:51 INFO DAGScheduler: Submitting 1 missing tasks from ResultStage 1 (PythonRDD[5] at takeOrdered at /app/query.py:60) (first 15 tasks are for partitions Vector(0))
26/04/03 14:21:51 INFO YarnScheduler: Adding task set 1.0 with 1 tasks resource profile 0
26/04/03 14:21:51 INFO TaskSetManager: Starting task 0.0 in stage 1.0 (TID 1) (cluster-slave-1, executor 1, partition 0, NODE_LOCAL, 8828 bytes)
26/04/03 14:21:51 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on cluster-slave-1:34587 (size: 6.7 KiB, free: 93.3 MiB)
26/04/03 14:21:52 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 0 to 172.25.0.2:57436
26/04/03 14:21:52 INFO TaskSetManager: Finished task 0.0 in stage 1.0 (TID 1) in 892 ms on cluster-slave-1 (executor 1) (1/1)
26/04/03 14:21:52 INFO YarnScheduler: Removed TaskSet 1.0, whose tasks have all completed, from pool
26/04/03 14:21:52 INFO DAGScheduler: ResultStage 1 (takeOrdered at /app/query.py:60) finished in 0.924 s
26/04/03 14:21:52 INFO DAGScheduler: Job 0 is finished. Cancelling potential speculative or zombie tasks for this job
26/04/03 14:21:52 INFO YarnScheduler: Killing all running tasks in stage 1: Stage finished
26/04/03 14:21:52 INFO DAGScheduler: Job 0 finished: takeOrdered at /app/query.py:60, took 6.752439 s
12719760        A Girl, Three Guys, and a Gun
11984610        A Catalogue of Crime
13375343        A Feud in the Kentucky Hills
13434748        A Day's Adventure
1466166 A Day at the Beach
13401464        A Gamble with Death
13387430        A Cry for Help (1912 film)
13391420        A Chance Deception
10849680        A Day in the Death of Donny B
14994019        A Fine, Windy Day
26/04/03 14:21:52 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/04/03 14:21:52 INFO SparkUI: Stopped Spark web UI at http://cluster-master:4040
26/04/03 14:21:52 INFO YarnClientSchedulerBackend: Interrupting monitor thread
26/04/03 14:21:52 INFO YarnClientSchedulerBackend: Shutting down all executors
26/04/03 14:21:52 INFO YarnSchedulerBackend$YarnDriverEndpoint: Asking each executor to shut down
26/04/03 14:21:52 INFO YarnClientSchedulerBackend: YARN client scheduler backend Stopped
26/04/03 14:21:52 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/04/03 14:21:52 INFO MemoryStore: MemoryStore cleared
26/04/03 14:21:52 INFO BlockManager: BlockManager stopped
26/04/03 14:21:52 INFO BlockManagerMaster: BlockManagerMaster stopped
26/04/03 14:21:52 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/04/03 14:21:52 INFO SparkContext: Successfully stopped SparkContext
26/04/03 14:21:53 INFO ShutdownHookManager: Shutdown hook called
26/04/03 14:21:53 INFO ShutdownHookManager: Deleting directory /tmp/spark-2cdd47ba-170c-4e43-908d-a6ce73ac90ab/pyspark-b1d55562-3e2f-47a3-a120-e9e9cf18f61a
26/04/03 14:21:53 INFO ShutdownHookManager: Deleting directory /tmp/spark-2cdd47ba-170c-4e43-908d-a6ce73ac90ab
26/04/03 14:21:53 INFO ShutdownHookManager: Deleting directory /tmp/spark-94b4a613-e798-4675-a87b-dfca3cb92c3a
26/04/03 14:21:53 INFO ShutdownHookManager: Deleting directory /tmp/localPyFiles-1c6b25cc-8e26-4213-b839-4901dcd2c9d0
jeanne@MacBook-Air--Jeanne bd_ass2_new %
```

2.6 Reflections on Results

The BM25 ranking behaves as expected across all tested queries. Documents containing multiple query terms with high term frequency rank at the top, while documents with only partial overlap appear lower in the list.

The `idf` component of the formula means that terms appearing in many of the 100 documents contribute less to the final score, while rare and specific terms have stronger discriminative power. This is visible in practice: a query like `"money banking"` returns documents specifically about finance and economics rather than general articles that happen to mention these words once.

The `b = 0.75` length normalization ensures that longer documents are not unfairly rewarded for simply containing more tokens. With an average document length of several hundred tokens across the corpus, this normalization has a noticeable effect on ranking order.

One limitation of the current setup is the small corpus size. With only 100 documents, some queries return fewer than 10 strongly relevant results, so the lower-ranked entries may have only marginal term overlap with the query. A larger corpus would produce sharper distinctions between relevant and non-relevant documents and make the BM25 scores more meaningful.

---

3. Conclusion

The assignment is complete. The project implements a mostly distributed search pipeline, with a local driver-based staging step in data preparation and fully distributed execution for all subsequent stages:

- **Data preparation** with PySpark, reading from a Parquet corpus and writing to HDFS

- **Inverted index construction** with two Hadoop MapReduce jobs, producing postings, vocabulary, document metadata, and corpus statistics
- **Index storage** in Cassandra, with a schema designed around the access patterns of BM25 retrieval
- **Query execution** with PySpark on YARN, using BM25 ranking to return the top 10 documents per query

The system starts with `docker compose up` and requires no manual steps. The output is reproducible and confirmed across a clean clone and full Docker run.